

LOGIC GATES AND BOOLEAN ALGEBRA

3.1 Introduction

There are mainly three fundamental operations performed in digital circuits: AND, OR, and NOT. These operations are called logic operations. There are also a few other logic operations like NAND, NOR, XOR, and XNOR.

A logic gate is an electronic circuit that performs a logic operation (AND, OR, NOT etc). These logic gates can be constructed using resistors, diodes, and transistors connected in such a way that the gate output is the result of a logic operation performed on the input signals. These gates are the building blocks of all digital circuits.

3.2 AND Operation

The logical AND operation produces a HIGH (1) output if and only if all the inputs are HIGH (1). If A and B represent two inputs variables, the logical AND operation, X can be expressed as

$$X = A \cdot B$$

The symbol '.' (dot) is used for logical AND operator. Here it is clear that X will be 1 if and only if A=1 and B=1.

The logical AND operation can be explained with a truth table. A truth table shows how a logic circuit's output depends on the inputs. Fig.1 (a) shows the truth table for a 2-input AND operator and Fig.1 (b) represents its logic symbol.

Truth table

Input		Output
A	B	$X = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

(a)



(b)

Fig.1: (a) Truth table for a 2-input AND operator
(b) Logic symbol for a 2-input AND gate

An AND gate is the physical realization of the logical AND operation. It is an electronic circuit which produces a HIGH output, only if all the inputs are also HIGH. The logic symbol for a 2-input AND gate is shown in Fig.1 (b).

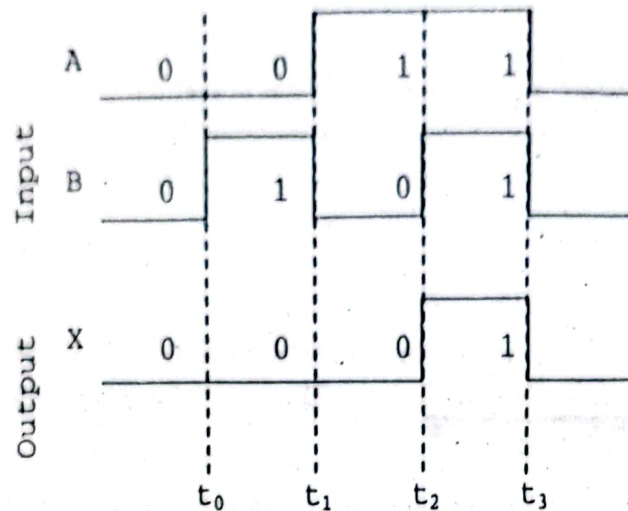
Pulsed operation:

Fig.2: Pulsed operation of a 2-input AND gate

As we known from the truth table that the output of an AND gate will be HIGH only if all the inputs are HIGH. Accordingly, during the interval t_2 - t_3 , both inputs A and B are HIGH, thereby producing a HIGH output. The logical expression for an N-input AND gate is

$$X = A_1 \cdot A_2 \cdot \dots \cdot A_N$$

and its logic symbol is shown in Fig. 3.

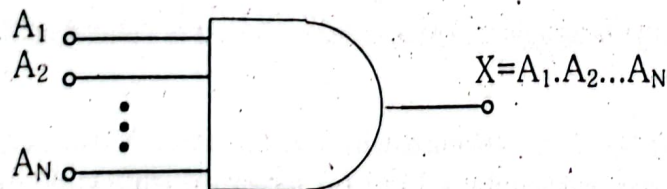


Fig.3. Logic symbol of an N-input AND Gate

3.3 OR Operation

The logical OR operation states that it produces a HIGH (1) output if at least one input is HIGH (1). If A and B are two input signals, the logical OR operation, X can be expressed as

$$X = A + B$$

The symbol '+' (plus) is used for logical OR operator. Here it is clear that $X=0$ if and only if all the inputs are 0 i.e. $A=B=0$. Fig.4 (a) shows the truth table for a 2-input OR operator and Fig.4 (b) presents its logic symbol.

Truth table

Input		Output
A	B	$X=A+B$
0	0	0
0	1	1
1	0	1
1	1	1

(a)



(b)

Fig.4: (a) Truth table for a 2-input OR operator
(b) Logic symbol for a 2-input OR gate

Pulsed operation:

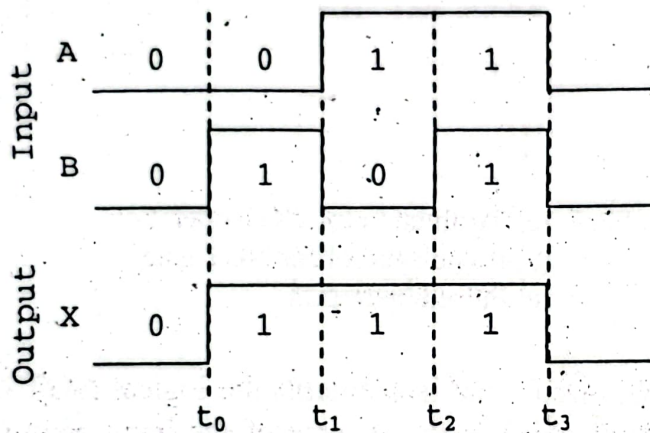


Fig.5: Pulsed operation of a 2-input OR gate

Fig.5 shows the Pulsed operation of a 2-input OR gate. It is seen that during the intervals t_0 to t_1 , t_1 to t_2 , and t_2 to t_3 , at least one input is HIGH (1), thereby producing a HIGH (1) output. Before t_0 , all inputs (A and B) are 0, thereby producing a 0 output.

Fig.6 shows the logical symbol of an OR gate having multiple inputs ($A_1, A_2, \dots, A_N$). Its logical equation is

$$X=A_1+A_2+\dots+A_N$$

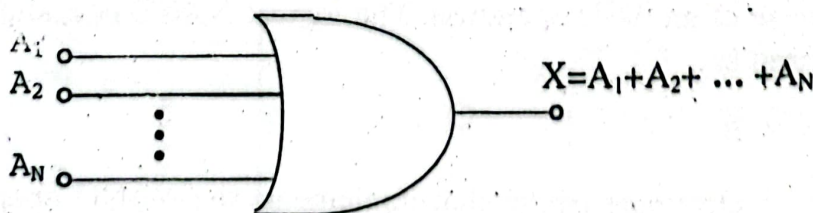


Fig.6. Logic symbol of an N-input OR gate

3.4 NOT Operation

The NOT operation is a unary operation which operates on a single variable. It produces an output which is opposite to the input, that is, the NOT operation inverts the input signal. The logical expression of NOT operation for input A can be expressed as

$$X = \bar{A}$$

The symbol ' - ' (over bar) is used for NOT operator. The NOT operation can be explained with the truth table and pulsed waveforms shown in Fig. 7(a) and 7(c).

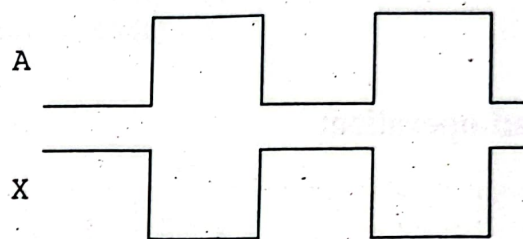
Truth table

Input	Output
A	$X = \bar{A}$
0	1
1	0

(a)



(b)



(c)

Fig.7: (a) Truth table for NOT operation
(b) Logic symbol for NOT gate
(c) Pulsed operation

The NOT gate is an electronic circuit that implements the logical NOT operation, that is, this gate generates an output signal which is the reverse of the input signal. Fig.7 (b) shows its logic symbol.

3.5 Some Other Logic Operations

So far we have discussed three fundamental logic operations: AND, OR, and NOT. These operations have been implemented by using AND, OR, and NOT gates, respectively. Although these three gates are sufficient to implement any logic expression, there are some other logic operations (NAND, NOR, XOR, and XNOR) occur frequently in digital systems, which can be derived from the basic gates (AND, OR, and NOT).

3.5.1 NAND Operation

The NAND operation states that it produces a HIGH output if at least one input is LOW. In fact, it does the reverse of an AND operation. The logical NAND operation for two inputs A and B can be expressed as

$$X = \overline{A \cdot B}$$

The NAND gate is an electronic circuit that implements the NAND operation. Fig.8 shows the truth table and logic symbol for a 2-input NAND gate.

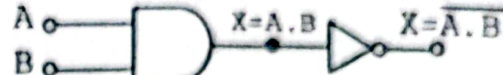
Truth table

Input		Output
A	B	$X = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

(a)



(b)



(c)

Fig.8: (a) Truth table for a 2-input NAND operator

(b) Logic symbol for a 2-input NAND gate

(c) Equivalent to an AND gate followed by a NOT gate

Pulsed operation:

Consider two waveforms for inputs A and B. The output of the NAND gate, X is shown below in Fig.9.

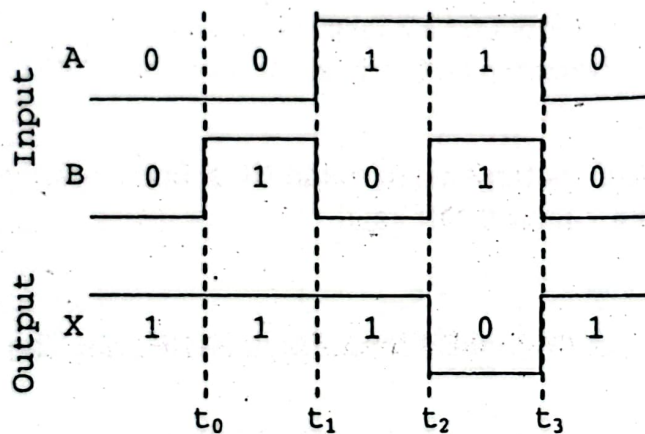


Fig.9: Pulsed operation of a 2-input NAND gate

For NAND gate, as we know that if all the inputs are HIGH, the output will be LOW. Accordingly, only during the interval between t_2 and t_3 , both inputs are HIGH, thereby producing a LOW output.

The output expression for an N-input NAND gate can be written as

$$X = \overline{A_1 \cdot A_2 \cdot \dots \cdot A_N}$$

and its logic symbol is shown below in Fig.10.

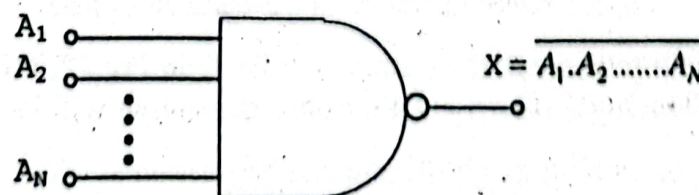


Fig.10. Logic symbol of an N-input NAND Gate

3.5.2 NOR Operation

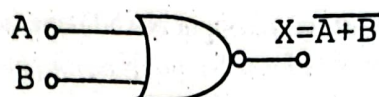
The NOR operation states that it produces a HIGH output, if and only if all the inputs are LOW. Otherwise, it produces a LOW output. It does the reverse of an OR operation. The logical NOR operation for two inputs A and B can be expressed as

$$X = \overline{A + B}$$

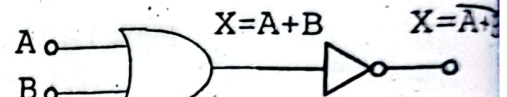
Truth table

Input		Output
A	B	$X = \overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0

(a)



(b)



(c)

Fig.11: (a) Truth table for a 2-input NOR operator
(b) Logic symbol for a 2-input NOR gate
(c) Equivalent to an OR gate followed by a NOT gate

The NOR gate is an electronic circuit that realizes the NOR operation. Fig.11 shows the truth table and the logic symbol for a 2-input NOR gate.

Pulsed operation:

Consider two waveforms for input A and B for a 2-input NOR gate. The output waveform, X, is drawn in Fig.12.

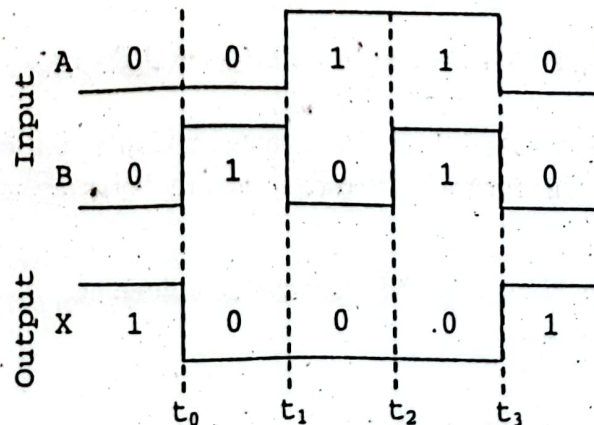


Fig.12: Pulsed operation of a 2-input NOR gate

For NOR gate, if all the inputs are 0, the output will be 1. In Fig.12, this occurs only before t_0 and after t_3 , thereby producing a 1 output. Otherwise, the output will be 0.

The output expression for an N-input NOR gate can be written as

$$X = \overline{A_1 + A_2 + \dots + A_N}$$

and its logic symbol is

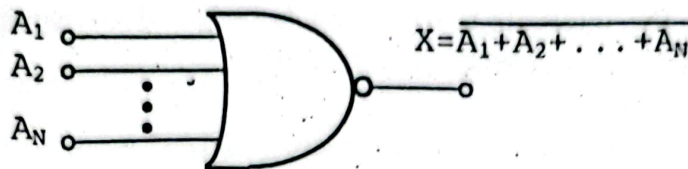


Fig.13. Logic symbol of an N-input NOR gate

3.5.3 XOR Operation

The XOR operation produces a HIGH (1) output whenever the two inputs are at opposite logic levels. Otherwise, the output is LOW (0). The logical XOR operation for two inputs A and B can be expressed as

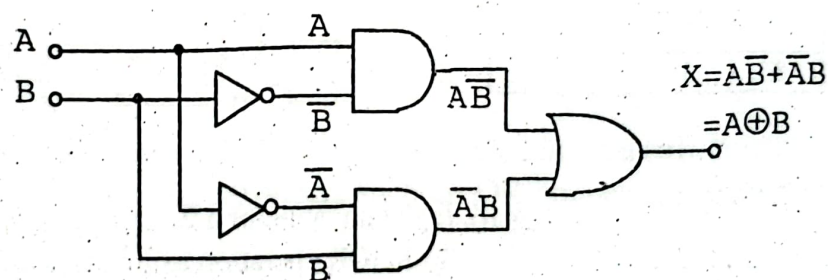
$$X = \overline{A}B + A\overline{B} = A \oplus B$$

The symbol ' $\oplus$ ' is used for XOR operator. Fig.14 (a) shows the truth table for a 2-input XOR operation. The logic XOR operation represented by the above expression can be implemented using the basic gates AND, OR, and NOT as shown in Fig. 14(b).

Truth table

Input		Output
A	B	$X = A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

(a)



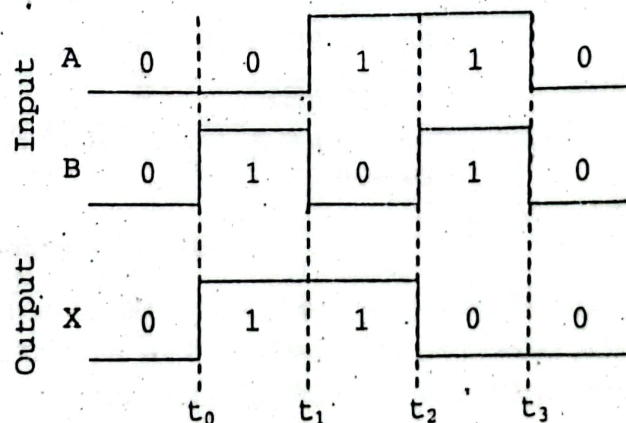
(b)

Fig.14: (a) Truth table for a 2-input XOR operator
(b) XOR operation using basic gates

Clearly, to implement the XOR operation, five basic gates (2 NOT gates, 2 AND gates, and 1 OR gate) are required. Fig.15 shows the logic symbol and pulsed operation of an XOR gate.



(a)



(b)

Fi. 15: (a) Logic symbol of a 2-input XOR gate
(b) Pulsed operation

In Fig.15 (b), only during the intervals t_0 to t_1 and t_1 to t_2 , inputs A and B are at opposite levels, thus producing HIGH ('1') outputs.

3.5.4 XNOR Operation

The XNOR operation produces a HIGH (1) output whenever the two inputs are at the same logic levels. Otherwise, the output is LOW (0). The logical XNOR operation for two inputs A and B can be expressed as

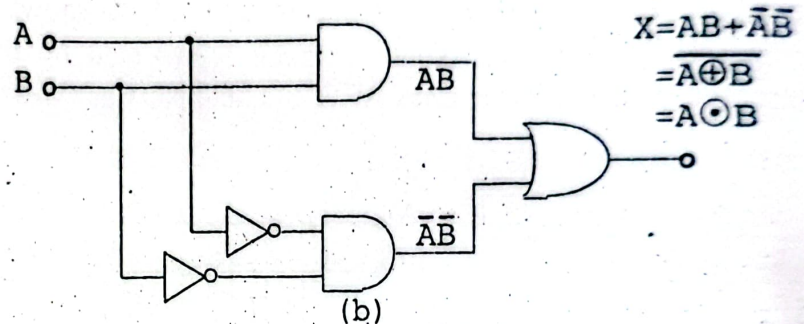
$$X = AB + \overline{A}\overline{B} = A \odot B$$

The symbol ' $\odot$ ' is used as XNOR operator. Fig. 16(a) shows the truth table of a 2-input XNOR operation and Fig. 16(b) presents the implementation of the 2-input XNOR operation using the basic gates AND, OR, and NOT.

Truth table

Input		Output
A	B	$X = A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

(a)

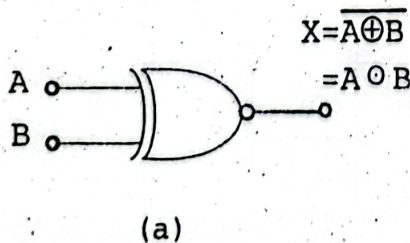


(b)

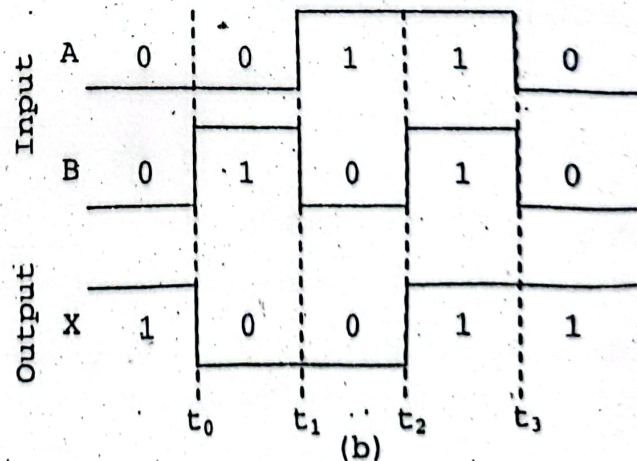
Fig.16: (a) Truth table for a 2-input XNOR operator
(b) XNOR operation using basic gates

Clearly, to implement the XNOR operation, five basic gates (2 NOT gates, 2 AND gates, and 1 OR gate) are required. In fact, an XNOR gate does the reverse operation of an XOR gate. Fig.17 shows the logic symbol and pulsed operation of an XNOR gate.

In Fig.17 (b), during the intervals before t_0 and after t_2 , both inputs A and B are at the same logic levels, thus producing HIGH ('1') outputs. Otherwise, its output is LOW.



(a)



(b)

Fig.17: (a) Logic symbol of a 2-input XNOR gate
(b) Pulsed operation

There is no multiple input XNOR gate.

3.6 Wired-Logic Operation

3.6.1 Wired-AND Operation

If the outputs of two or more NAND gates are tied together, an additional AND operation is performed without using a real AND gate. Fig.18 shows such an AND connection. This type of connection is also called Wired-AND or Dotted-AND or Implied-AND.

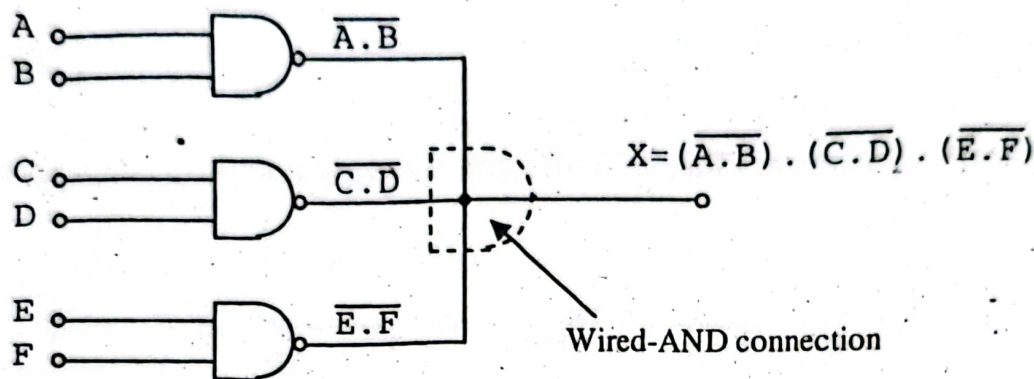


Fig.18: Wired-AND connection

3.6.2 Wired-OR Operation

If the outputs of two or more NOR gates are tied together, an additional OR operation is performed without using a real OR gate. Fig.19 shows such an OR connection. This type of connection is called Wired-OR connection.

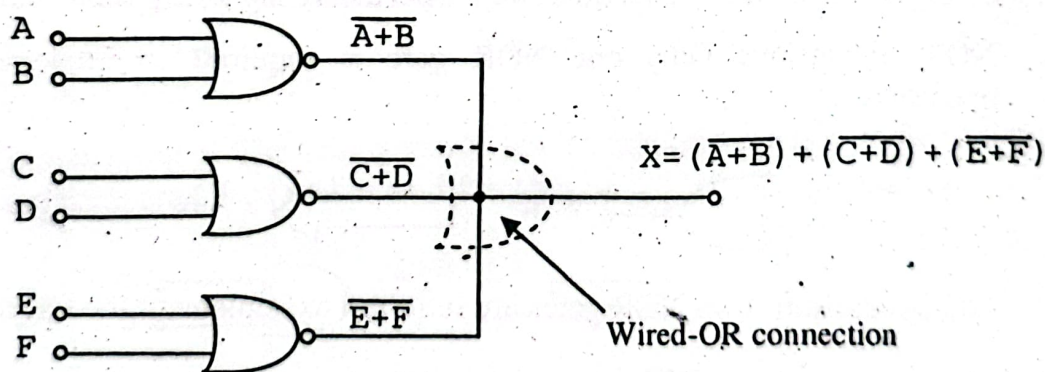


Fig.19: Wired-OR connection

3.7 The Universal Property of NAND and NOR Gates

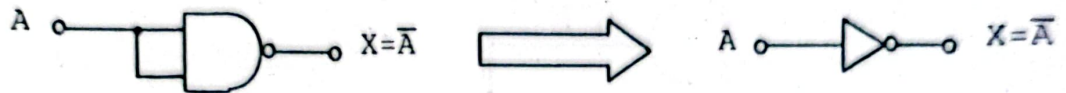
In digital systems, any logic expression can be implemented by using a combination of the basic gates: AND, OR, and NOT gates. That is, if we have a sufficient number of AND, OR, and NOT gates, any digital circuit can be designed. But the shortage of any one of these basic gates makes the circuit impossible to be designed.

On the other hand, it is also possible to design any digital circuit by using either only NAND gates or only NOR gates. That is, if we have a sufficient number of NAND or NOR gates

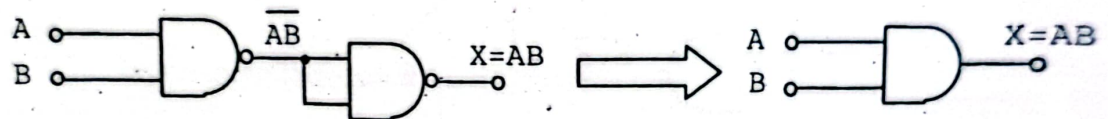
only, we can implement the desired circuit. No other gates (AND, OR, NOT) are required. This is why NAND and NOR gates are called universal gates.

A. Implementation of AND, OR, and NOT operations by using only NAND gates:

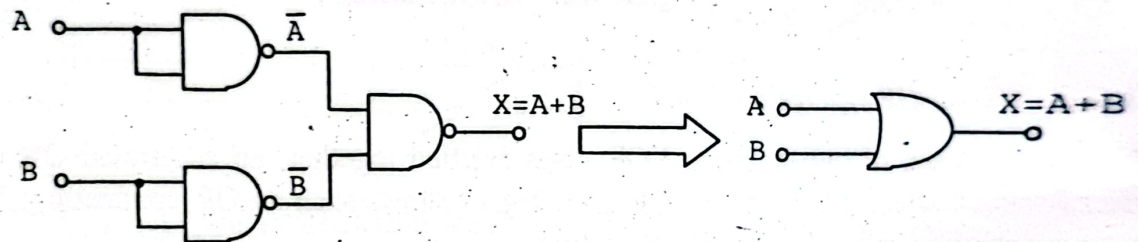
- i. **NOT Operation:** Only one NAND gate is sufficient to implement the NOT operation.



- ii. **AND operation:** Two NAND gates are required to implement the AND operation.

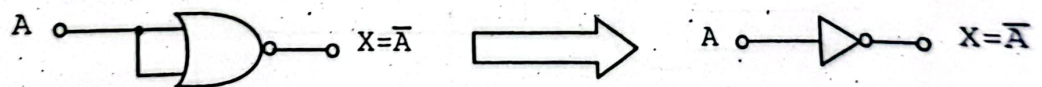


- iii. **OR operation:** Three NAND gates are required to implement the OR operation.

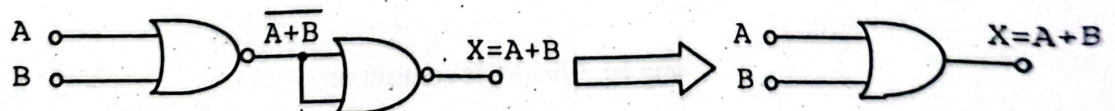


B. Implementation of AND, OR, and NOT operations by using only NOR gates:

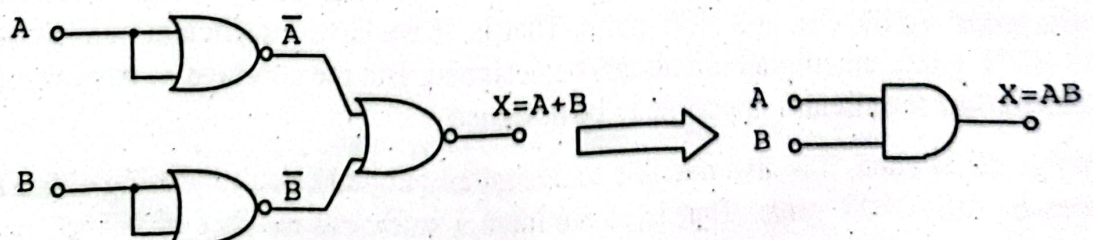
- i. **NOT operation:** Only one NOR gate is required to implement the NOT operation.



- ii. **OR operation:** Two NOR gates are required to implement the OR operation.



- iii. **AND operation:** Three NOR gates are required to implement the AND operation.



Here we see that the fundamental logic operations: AND, OR, and NOT have been implemented using only either NAND or NOR gates. Therefore, any logic circuit can be designed using only NAND or NOR gates.

3.8 Introduction to Boolean Algebra

In 1854, an English mathematician George Boole (1815-1864) developed some rules for manipulating binary variables known as Boolean algebra. But Boolean algebra had no practical applications until 1938. It was then known as Boolean Postulates. In 1938, Claude E. Shannon proposed the use of Boolean algebra in the design of relay switching circuits. Because of the similarities of operations between relays and electronic circuits, Boolean algebra is used extensively as a tool for the analysis and design of digital systems.

3.9 Laws and Rules of Boolean Algebra

Laws and Rules of Boolean algebra are useful in simplifying logic expressions. A logic expression determines the number of gates required to obtain the desired output. It gives an idea about the type of gates needed and their required connections in the circuit. The more complex an expression, the more complex the circuit. To make the circuit simple, it is therefore necessary to simplify the expression. Boolean laws and rules are used for this purpose.

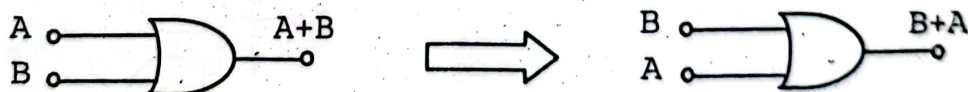
3.9.1 Laws of Boolean Algebra

The basic laws of Boolean algebra are discussed below:

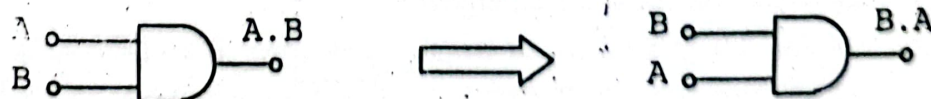
1. Commutative Laws

- i) $A+B=B+A$
- ii) $A.B=B.A$

Explanation: i) The commutative law of addition states that the order in which the variables are ORed makes no difference i.e.



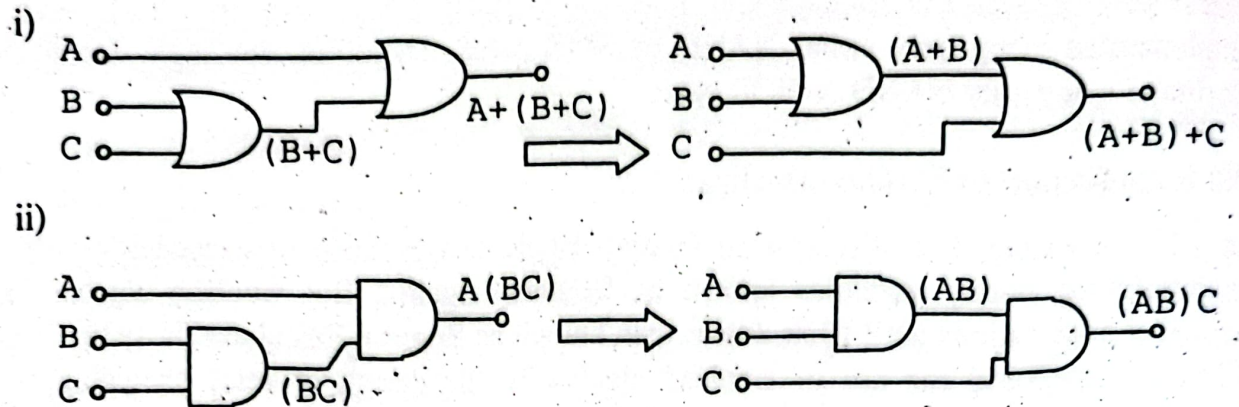
ii) The commutative law of multiplication states that the order in which the variables are ANDed makes no difference i.e.



2. Associative Laws

- i) $A+(B+C)=(A+B)+C$
- ii) $A.(B.C)=(A.B).C$

Explanation: The associative laws state that we can group the variables in an OR expression or AND expression in any order i.e.

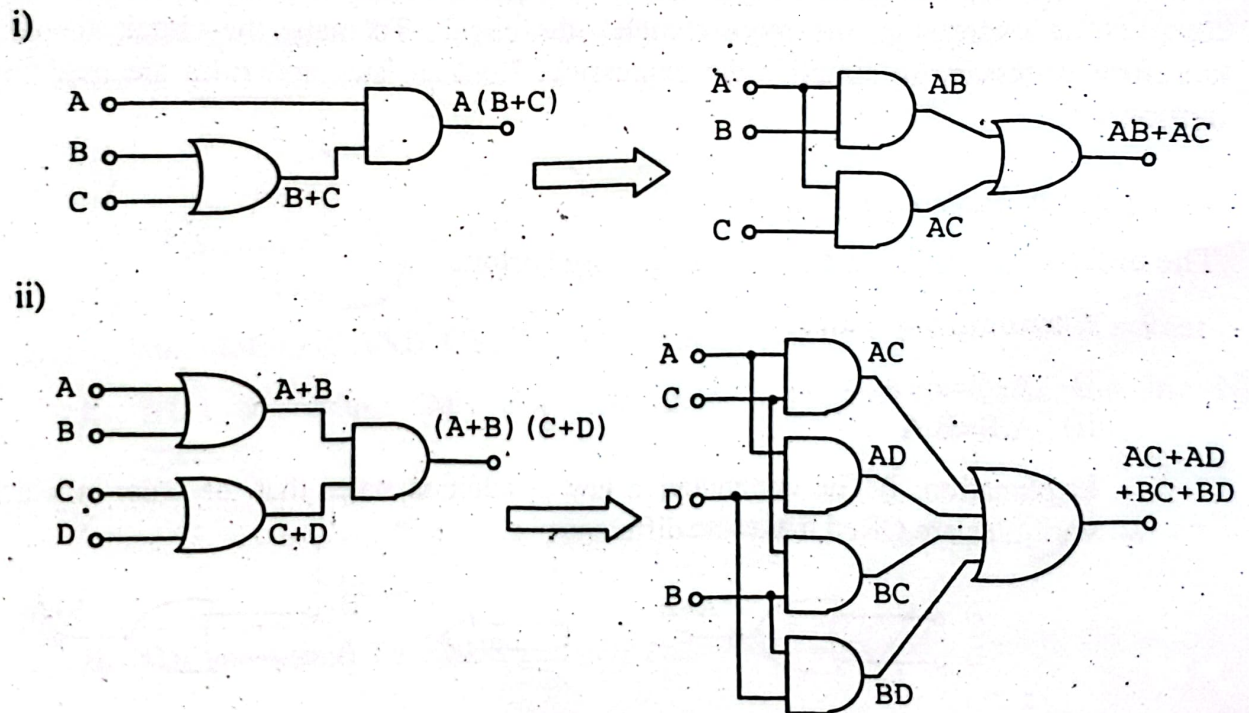


3. Distributive Laws

i) $A \cdot (B + C) = A \cdot B + A \cdot C$

ii) $(A + B) \cdot (C + D) = A \cdot C + A \cdot D + B \cdot C + B \cdot D$

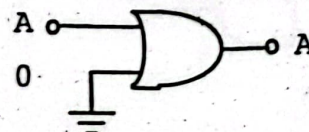
Explanation: The distributive laws state that we can expand an expression by multiplying term by term as like as in ordinary algebra or we can factor an expression i.e.



3.9.2 Basic Rules of Boolean Algebra

Following are the basic rules of Boolean algebra:

1. $A + 0 = A$



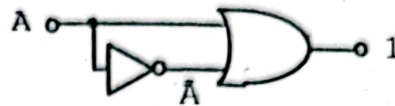
2. $A + 1 = 1$



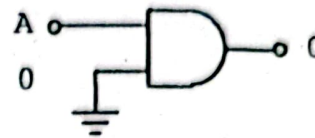
3. $A + A = A$



4. $A + \bar{A} = 1$



5. $A \cdot 0 = 0$



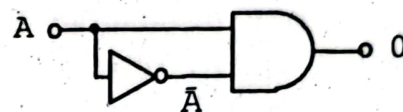
6. $A \cdot 1 = A$



7. $A \cdot A = A$

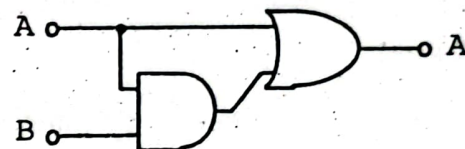


8. $A \cdot \bar{A} = 0$



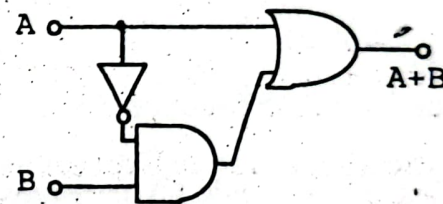
9. $A + A \cdot B = A$

Proof: $A + A \cdot B$
 $= A \cdot (1 + B)$
 $= A \cdot 1$
 $= A$



10. $A + \bar{A} \cdot B = A + B$

Proof: $A + \bar{A} \cdot B$
 $= A \cdot 1 + \bar{A} \cdot B$
 $= A \cdot (1 + B) + \bar{A} \cdot B$
 $= A + A \cdot B + \bar{A} \cdot B \quad [\because 1 + B = 1]$
 $= A + B(A + \bar{A})$
 $= A + B \cdot 1 \quad [\because A + \bar{A} = 1]$
 $= A + B$



3.10 De Morgan's Theorems

De Morgan, a French mathematician proposed two theorems, which are extensively used in simplifying logic expressions.

Theorem 1:

The complement of a product of variables is equal to the sum of the complements of the individual variables. For two variables A and B, the theorem can be expressed as

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

For n variables $X_1, X_2, X_3, \dots, X_n$, the theorem can be expressed as

$$\overline{X_1 \cdot X_2 \cdot X_3 \dots X_n} = \bar{X}_1 + \bar{X}_2 + \bar{X}_3 + \dots + \bar{X}_n$$

Theorem 2:

The complement of a sum of variables is equal to the product of the complements of the individual variables. For two variables A and B, the theorem can be expressed as

$$\overline{A+B} = \overline{A} \cdot \overline{B}$$

For n variables $X_1, X_2, X_3, \dots, X_n$, the theorem can be expressed as

$$\overline{X_1 + X_2 + X_3 + \dots + X_n} = \overline{X_1} \cdot \overline{X_2} \cdot \overline{X_3} \dots \overline{X_n}$$

3.11 Principle of Duality

In Boolean algebra there is a precise duality between AND (.) and OR (+) operators, and the digits 0 and 1. That is, any theorem in Boolean algebra has its dual obtainable by interchanging i) '+' with '.' and ii) '0' with '1'. Hence, if a particular theorem is proved, its dual theorem is automatically holds, and need not be proved separately. This property is known as the principle of duality in Boolean algebra. For example, consider rule 2:

$$A+1=1$$

Therefore, its duality can be obtained by interchanging '+' with '.' and '1' with '0' as given below:

$$A \cdot 0 = 0$$

This is nothing but rule 5 (Section 3.9.2)

3.12 Simplification of Logic Expressions using Boolean Algebra

Any logic expression can be simplified using the Boolean algebraic theorems. But we can not say whether the simplified expression is in the full simplest form or it can be simplified further. Simplification using Boolean algebra is normally done in two steps:

1. Convert the original expression into Sum-of-products form.
2. Check the product terms and then factoring is performed wherever possible.

Example 1. Simplify the following expression and draw the logic diagram.

$$\overline{A}\overline{B}C(BD+CDE) + \overline{A}\overline{C}$$

Solution: $\overline{A}\overline{B}C(BD+CDE) + \overline{A}\overline{C}$

$$= \overline{A}\overline{B}C.BD + \overline{A}\overline{B}C.CDE + \overline{A}\overline{C}$$

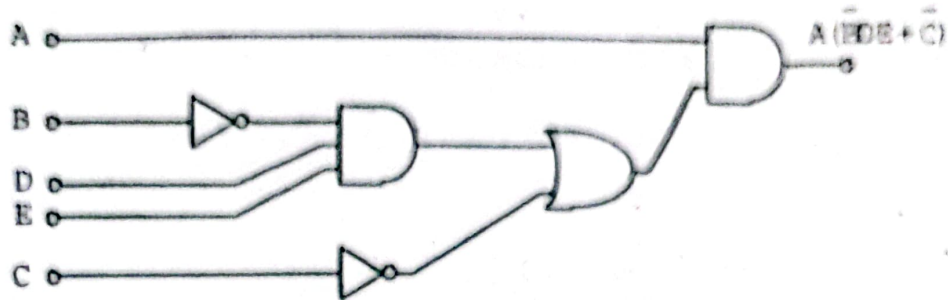
$$= \overline{A}CDB.B + \overline{A}\overline{B}CDE + \overline{A}\overline{C}$$

$$= 0 + \overline{A}\overline{B}CDE + \overline{A}\overline{C} \quad [\because \overline{B}.B = 0]$$

$$= \overline{A}(\overline{B}CDE + \overline{C})$$

$$= \overline{A}(\overline{B}DE + \overline{C}) \quad [\text{Rule 10, (Section 3.9.2)}]$$

Logic diagram:



Example 2. Simplify the following expression and draw the logic diagram.

$$AB + A(B + C) + B(B + C)$$

Solution: $AB + A(B + C) + B(B + C)$

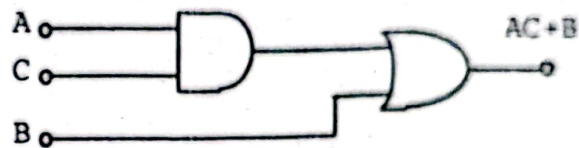
$$= AB + AB + AC + BB + BC$$

$$= AB + AC + B + BC$$

$$= (A + 1 + C)B + AC$$

$$= B + AC$$

Logic diagram:



Example 3. Simplify the following expression and draw the logic diagram.

$$\overline{(A + \overline{B})}.B.(A + \overline{C})$$

Solution: $\overline{(A + \overline{B})}.B.(A + \overline{C})$

$$= (\overline{A.B}).B.(A + \overline{C})$$

[Applying De Morgan's theorem, $\overline{A + B} = \overline{A}.\overline{B}$]

$$= \overline{A}.B.B.(A + \overline{C})$$

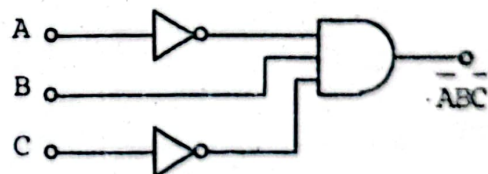
$$= \overline{A}.B.(A + \overline{C})$$

$$= \overline{A}.B.A + \overline{A}.B.\overline{C}$$

$$= 0 + \overline{A}B\overline{C}$$

$$= \overline{A}B\overline{C}$$

Logic diagram:



Example 4. Simplify the following expression and draw the logic diagram.

$$\overline{A}C(\overline{A}BD) + \overline{A}BCD + \overline{A}BC$$

Solution: $\overline{A}C(\overline{A}BD) + \overline{A}BCD + \overline{A}BC$

$$= \overline{A}C(\overline{A} + \overline{B} + \overline{D}) + \overline{A}BCD + \overline{A}BC$$

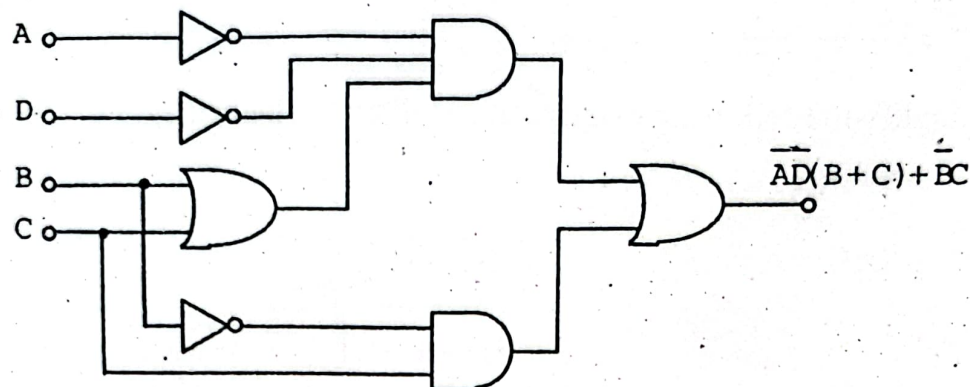
$$= \overline{A}C(A + \overline{B} + \overline{D}) + \overline{A}BCD + \overline{A}BC$$

$$= \bar{A}CA + \bar{A}CB + \bar{A}CD + \bar{A}BCD + \bar{A}BC$$

$$= 0 + \bar{B}C(\bar{A} + A) + \bar{A}D(C + \bar{B}C) \quad [\because \bar{A}.A = 0]$$

$$= \bar{B}C + \bar{A}D(B + C) \quad [\because \bar{A}.A = 1 \text{ and } \bar{B}C + C = B + C]$$

Logic diagram:



Example 5. Simplify the following expression and draw the logic diagram.

$$(B + \bar{C})(\bar{B} + C) + \bar{A} + B + \bar{C}$$

Solution: $(B + \bar{C})(\bar{B} + C) + \bar{A} + B + \bar{C}$

$$= B\bar{B} + BC + \bar{B}\bar{C} + \bar{C}C + \bar{A} + B + \bar{C}$$

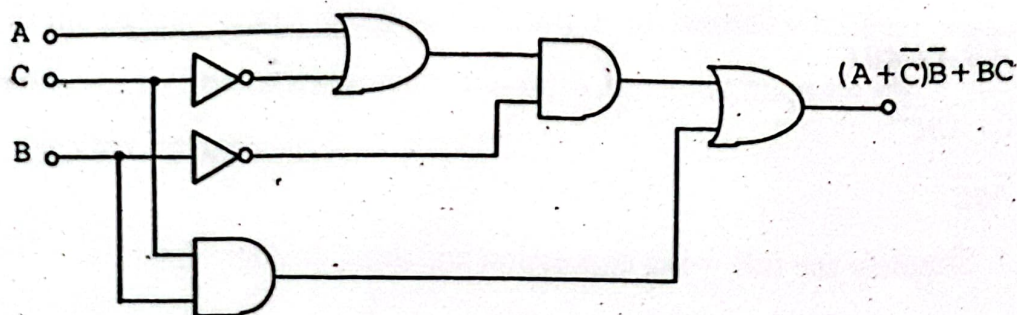
$$= 0 + BC + \bar{B}\bar{C} + 0 + \bar{A} + B + \bar{C}$$

$$= BC + \bar{B}\bar{C} + \bar{A} + B + \bar{C}$$

$$= BC + \bar{B}(\bar{C} + AC)$$

$$= BC + \bar{B}(A + \bar{C}) \quad [\because AC + \bar{C} = A + \bar{C}]$$

Logic circuit:



Example 6. Simplify the following expression.

$$[\bar{A}B(C + BD) + \bar{A}B]C$$

Solution: $[\bar{A}B(C + BD) + \bar{A}B]C$

$$= [\bar{A}BC + \bar{A}BBD + \bar{A}B]C$$

$$\begin{aligned}
 &= [\overline{A}B\overline{C} + 0 + \overline{A}B]C \\
 &= \overline{A}B\overline{C}.C + \overline{A}B.C \\
 &= \overline{A}B\overline{C} + \overline{A}BC \\
 &= (A + \overline{A})\overline{B}C \\
 &= \overline{B}C
 \end{aligned}$$

Example 7. Simplify the expression: $\overline{A}B + A(\overline{B+C}) + B(\overline{B+C})$

Solution: $\overline{A}B + A(\overline{B+C}) + B(\overline{B+C})$

$$\begin{aligned}
 &= \overline{A}B + A.\overline{B}.\overline{C} + B.\overline{B}.\overline{C} \\
 &= \overline{A}B + A.\overline{B}.\overline{C} + 0 \\
 &= \overline{A}B(1 + \overline{C}) \\
 &= \overline{A}B
 \end{aligned}$$

Example 8. Simplify the expression: $[AB(C + \overline{B}D) + \overline{A}B]CD$.

Solution: $[AB(C + \overline{B}D) + \overline{A}B]CD$

$$\begin{aligned}
 &= [AB(C + \overline{B} + \overline{D}) + \overline{A} + \overline{B}]CD \\
 &= ABCCD + AB\overline{B}CD + AB\overline{D}CD + \overline{A}CD + \overline{B}CD \\
 &= ABCD + 0 + 0 + \overline{A}CD + \overline{B}CD \\
 &= ABCD + \overline{A}CD + \overline{B}CD \\
 &= (AB + \overline{A})CD + \overline{B}CD \\
 &= (\overline{A} + B)CD + \overline{B}CD \\
 &= (\overline{A} + B + \overline{B})CD \quad [\because \overline{A} + B + \overline{B} = \overline{A} + 1 = 1] \\
 &= CD
 \end{aligned}$$

Example 9. Simplify the expression: $(\overline{A}C)(B + \overline{C})\overline{B}$

Solution: $(\overline{A}C)(B + \overline{C})\overline{B}$

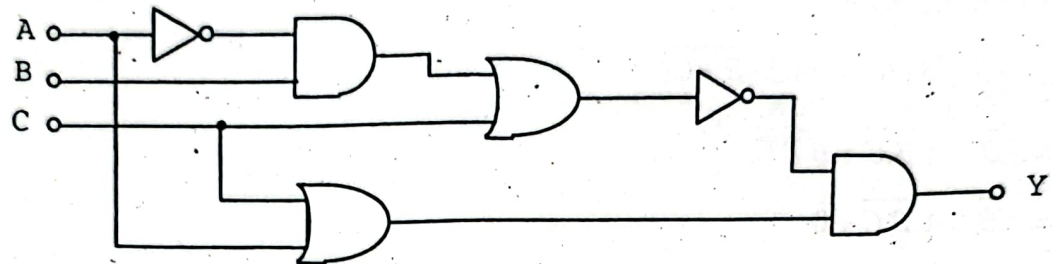
$$\begin{aligned}
 &= (\overline{A} + C)(B + \overline{C})\overline{B} \\
 &= \overline{A}B\overline{B} + \overline{A}C\overline{B} + BC\overline{B} + C\overline{C}B \\
 &= 0 + \overline{A}C\overline{B} + 0 + 0 \\
 &= \overline{A}C\overline{B} \\
 &= \overline{A+B+C}
 \end{aligned}$$

3.13 Simplification of Logic Circuits

Simplification of logic circuits can easily be done using the following steps:

- Derive the output logic expression of the given circuit,
- Simplify it, and finally
- Realize the simplified expression.

Example 10. Simplify the following logic circuit.



Solution: $Y = \overline{\overline{AB} + C}(A + C)$

$$= (\overline{\overline{A} + \overline{B}})\overline{C}(A + C)$$

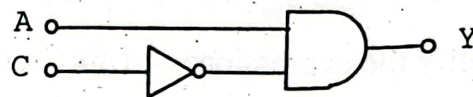
$$= (A + B)(\overline{C}A + \overline{C}C)$$

$$= A\overline{C}A + B\overline{C}A$$

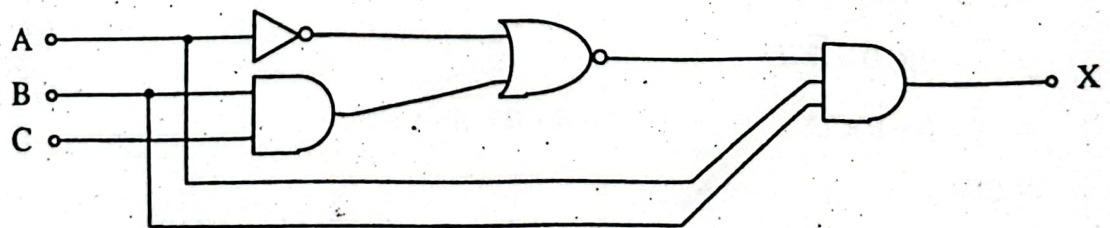
$$= (1 + B)\overline{C}A$$

$$= A\overline{C}$$

After simplification, the logic circuit is



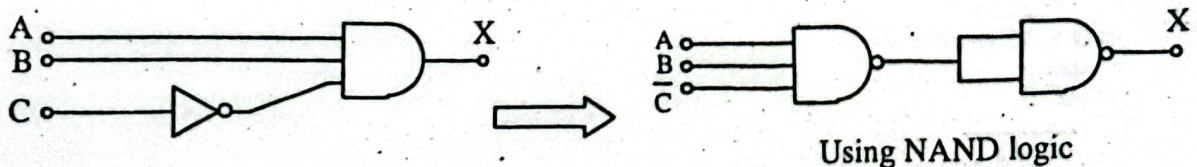
Example 11. Simplify the following logic circuit and implement using only NAND gates.



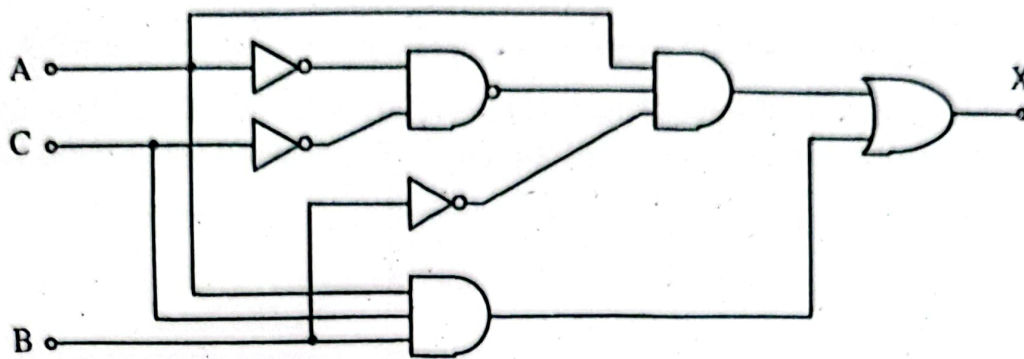
Solution: The logic expression for the output, X is

$$X = \overline{\overline{A + BC}}AB = \overline{\overline{A} \cdot \overline{BC}}AB = A \cdot \overline{BC} \cdot AB = AB(\overline{B} + \overline{C}) = ABC$$

Therefore, the simplified logic circuit is,



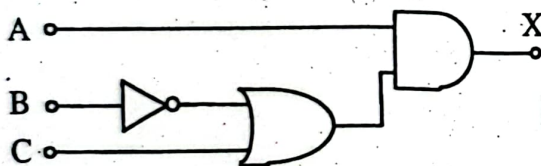
Example 12. Simplify the following logic circuit and implement using only NAND gates.



Solution: The logic expression for the output, X is

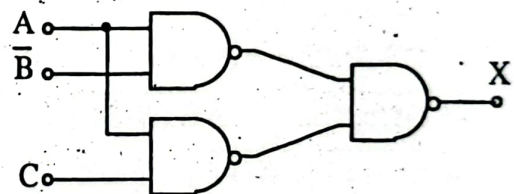
$$\begin{aligned}
 X &= A(\overline{AC})\overline{B} + ABC \\
 &= A(\overline{A} + \overline{C})\overline{B} + ABC = A(\overline{A} + \overline{C})\overline{B} + ABC = \overline{A}\overline{B} + \overline{A}\overline{B}C + ABC \\
 &= \overline{A}\overline{B}(1 + C) + ABC = \overline{A}\overline{B} + ABC = A(\overline{B} + BC) = A(\overline{B} + C)
 \end{aligned}$$

Therefore, the simplified logic circuit is



$$X = A(\overline{B} + C) = \overline{A}\overline{B} + AC$$

$$= \overline{\overline{A}\overline{B}} \cdot \overline{\overline{AC}} = (\overline{AB})(\overline{AC})$$



Using NAND logic

REVIEW QUESTIONS

1. Describe the logical operation of an AND/OR/NAND/NOR/XOR/XNOR gate with timing diagram.
2. "AND, OR, and NOT gates are logically complete." Discuss briefly.
3. Discuss Wired-AND/Wired-OR operation.
4. Why are NAND and NOR gates called universal gates?
5. Prove the universality of NAND and NOR gates.
6. Show the implementations of the logical operations AND, OR, and NOT (i) with only NAND gates and (ii) with only NOR gates.

i. $Y = (\overline{ABC})(\overline{A+B+C})$

ii. $Y = \overline{A}(\overline{B+C})(A+B+\overline{C})$

iii. $Y = (\overline{A+B})(A+B+D)\overline{D}$

iv. $Y = (\overline{A \oplus B}) + BC(A+B)$

v. $Y = (A+BC)(\overline{D+BC})\overline{AD}$

vi. $Y = (A+B)(A+\overline{AB})C + \overline{AB} + ABC + \overline{\overline{ABC}}$

18. Develop a truth table from the output expression for the following logic circuit:

